

v0.1 Trace数据库设计方案

1.模块概述（技术需求分析）

Trace 模块是 v0.1 的可观测性基础设施，目标是对每一次 LLM 抽取任务进行完整记录，确保任何阶段的失败都有迹可查，任何抽取结果都可以追溯到具体的执行过程。Trace 数据同时服务于三个目的：故障复盘（发生错误时找原因）、质量评估（统计抽取准确率和 token 消耗）、版本对比（v0.2 多模型对比的数据基础）。

Trace 模块需记录以下六类信息：每步的输入（prompt_system、prompt_user）、每步的输出（raw_response、parsed_output）、**LLM 对核心字段的判断理由**、每次 API 调用的模型名称、token 数量（input/output）、响应时间。

2.技术目标

- Trace 写入成功率100%
- token 汇总准确性100%
- LLM 判断理由记录完整率100%

3.方案设计

3.1 数据库表

Code block

```
1  //
  //
  //
2  // Trace 数据库字段定义
3  // hap_trace.db · v0.1
4  //
  //
  //
5
6  {
7  //
```

```

8      // 表一: extraction_runs
9      // 记录每次抽取任务的整体信息
10     // 一篇论文 = 一条记录, 无论中间调用了几次 LLM
11     //

```

```

12     "extraction_runs": {
13
14         // — 标识

```

```

15         "run_id": {
16             "type": "TEXT",
17             "constraint": "PRIMARY KEY",
18             "format": "run_YYYYMMDD_HHMMSS_xxxxxx",
19             "definition": "本次抽取任务的唯一标识, 时间戳 + UUID hex 前 6 位组合, 全局唯一。run_id 在 __init__() 时生成, 但 INSERT DB 在 start() 时执行"
20         },
21
22         "paper_id": {
23             "type": "TEXT",
24             "constraint": "NOT NULL",
25             "format": "P000001",
26             "definition": "关联业务数据库 paper_record.paper_id, 标识本次任务处理的是哪篇
论文"
27         },
28
29         // — 配置信息

```

```

30         "model_name": {
31             "type": "TEXT",
32             "constraint": "NOT NULL",
33             "allowed_values": ["gpt-4o", "claude-opus-4-6"],
34             "definition": "本次任务实际调用的模型名称, v0.1 固定为 gpt-4o, v0.2 多模型对比
时会有不同值"
35         },
36
37         "prompt_version": {
38             "type": "TEXT",
39             "constraint": "NOT NULL",
40             "format": "v0.1.0",
41             "definition": "System Prompt 的版本号, 用于追踪 Prompt 变更对抽取质量的影响,
修改 Prompt 时必须更新此版本号"
42         },
43
44         "schema_version": {
45             "type": "TEXT",

```

```
46     "constraint": "NOT NULL",
47     "format": "v0.05",
48     "definition": "字段字典（field_dict.json）的版本号，用于追踪字段字典变更对抽取
质量的影响"
49 },
50
51 // — 时间

```

```
52     "started_at": {
53         "type": "TEXT",
54         "format": "ISO8601 UTC",
55         "definition": "任务开始时间。与 created_at 分开：created_at 是 DB 写入时间，
started_at 是业务意义上的任务开始时间"
56     },
57
58     "finished_at": {
59         "type": "TEXT",
60         "format": "ISO8601 UTC",
61         "definition": "任务结束时间，无论成功还是失败，finalize() 调用时写入"
62     },
63
64 // — Token 汇总

```

```
65     "total_input_tokens": {
66         "type": "INTEGER",
67         "default": 0,
68         "definition": "本次 run 所有 trace_steps 的 input_tokens 之和，由
finalize() 从 trace_steps 聚合计算，不依赖调用方传入"
69     },
70
71     "total_output_tokens": {
72         "type": "INTEGER",
73         "default": 0,
74         "definition": "本次 run 所有 trace_steps 的 output_tokens 之和，聚合计算"
75     },
76
77     "total_tokens": {
78         "type": "INTEGER",
79         "default": 0,
80         "definition": "total_input_tokens + total_output_tokens，用于核算单篇论文的
总 token 消耗和成本"
81     },
82
83 // — 结果统计

```

```
84     "records_extracted": {
85         "type": "INTEGER",
```

```

86         "default": 0,
87         "definition": "LLM 输出并通过 Schema 校验的 record 数量, 由
len(parsed['records']) 计算, 不是数据库实际条数"
88     },
89
90     "records_inserted": {
91         "type": "INTEGER",
92         "default": 0,
93         "definition": "实际成功写入业务数据库的 record 数量, 正常情况下与
records_extracted 相等, 不等时说明写入过程有部分失败"
94     },
95
96     // — 完整性检查

```

```

97     "integrity_check_status": {
98         "type": "TEXT",
99         "allowed_values": ["passed", "failed", "skipped"],
100        "definition": "业务写入完成后对数据库进行反向校验的结果。passed = 三项检查
(record 数量、function_evidence 完整性、核心字段非 null) 全部通过"
101    },
102
103    "integrity_check_detail": {
104        "type": "TEXT",
105        "definition": "完整性检查失败时的具体原因描述, 如: 期望 3 条, 实际写入 2 条。
passed 时为 NULL"
106    },
107
108    // — 运行状态

```

```

109    "run_status": {
110        "type": "TEXT",
111        "constraint": "NOT NULL",
112        "default": "running",
113        "allowed_values": ["running", "success", "failed"],
114        "definition": "任务整体状态。running = 初始占位; success = 抽取、写入、完整性检
查全部通过; failed = 任意阶段失败"
115    },
116
117    "error_message": {
118        "type": "TEXT",
119        "definition": "run 级别的顶层错误信息, 描述整个任务失败的原因。step 级别的错误记
录在 trace_steps.error_detail, 粒度更细"
120    },
121
122    // — 系统元数据

```

```

123    "created_at": {

```

```

124         "type": "TEXT",
125         "default": "datetime('now')",
126         "definition": "数据库记录创建时间, SQLite 自动写入"
127     }
128 },
129
130
131 //

```

```

132 // 表二: trace_steps
133 // 记录每次 LLM API 调用的详细信息
134 // v0.1 正常情况下一个 run 对应一条记录
135 // v0.2 引入 repair / 分块后一个 run 可能对应多条记录
136 //

```

```

137     "trace_steps": {
138
139         // — 标识

```

```

140         "step_id": {
141             "type": "TEXT",
142             "constraint": "PRIMARY KEY",
143             "format": "run_xxx_step_01",
144             "definition": "单次 API 调用的唯一标识, 由 run_id + step_counter 拼接生成"
145         },
146
147         "run_id": {
148             "type": "TEXT",
149             "constraint": "NOT NULL, FOREIGN KEY → extraction_runs.run_id ON DELETE
CASCADE",
150             "definition": "关联 extraction_runs.run_id, 标识本条 step 属于哪次 run。ON
DELETE CASCADE: 删除 run 时对应 step 自动级联删除"
151         },
152
153         "step_index": {
154             "type": "INTEGER",
155             "constraint": "NOT NULL",
156             "definition": "同一 run 内的步骤序号, 从 1 开始自增。v0.1 只有 step_index =
1; v0.2 repair 调用时 step_index = 2"
157         },
158
159         "step_type": {
160             "type": "TEXT",
161             "default": "extraction",
162             "allowed_values": ["extraction", "repair"],

```

```
163         "definition": "本次调用的类型。extraction = 正常抽取调用；repair = v0.2 的修复  
调用（v0.1 不使用，预留字段）"  
164     },  
165  
166     // — 输入  


---

  
167     "prompt_system": {  
168         "type": "TEXT",  
169         "definition": "System Prompt 的完整内容，包含任务指令和字段字典 JSON，完整保存  
不截断，用于复盘不同版本 Prompt 对抽取质量的影响"  
170     },  
171  
172     "prompt_user": {  
173         "type": "TEXT",  
174         "definition": "User Prompt 的完整内容，包含论文全文，完整保存不截断"  
175     },  
176  
177     "input_tokens": {  
178         "type": "INTEGER",  
179         "definition": "本次调用的输入 token 数，包含 prompt_system 和 prompt_user 的  
全部内容，来自 response.usage.prompt_tokens"  
180     },  
181  
182     // — 输出  


---

  
183     "raw_response": {  
184         "type": "TEXT",  
185         "definition": "LLM 原始输出，完整字符串，无论解析成功与否都保存。是  
parse_error 故障复盘的核心依据，也是可以重新解析的原始数据"  
186     },  
187  
188     "output_tokens": {  
189         "type": "INTEGER",  
190         "definition": "本次调用的输出 token 数，来自  
response.usage.completion_tokens"  
191     },  
192  
193     // — 解析结果  


---

  
194     "parsed_output": {  
195         "type": "TEXT",  
196         "definition": "JSON 解析成功后的结构化输出，序列化为字符串存储。解析失败时为  
NULL。可以直接反序列化用于问题分析"  
197     },  
198  
199     "llm_reasoning": {  
200         "type": "TEXT",
```

```
201     "format": "JSON 字符串",
202     "definition": "LLM 对所有核心枚举字段的判断理由，从
parsed_output['reasoning'] 提取后序列化存储。包含每个有判断空间的核心字段（枚举字段、溯源
源字段）的推理依据，是人工复核定位原文的关键依据。解析失败时为 NULL",
203     "example": {
204         "entity_type_reason": "论文第2页明确写 short peptide，氨基酸数量12个，判定
为 peptide",
205         "source_origin_reason": "序列截取自釉原蛋白第42-54位，
TXT[p.3|Methods|para.1]，判定为 natural",
206         "effect_direction_reason": "Fig.2A 显示 HAp 结合量显著高于对照组，判定为
promote",
207         "trace_status_reason": "sequence、method、function、evidence 四个溯源字段
均有明确锚点，判定为 complete"
208     }
209 },
210
211 // — 时间

```

```
212 // 注意: called_at 和 response_at 均在收到响应后才写入 DB
213 // 原因: 避免 DB 写入操作本身的耗时（几毫秒）干扰 response_time_ms 的计算
214 // 两个时间戳在内存中用 time.time() 记录，收到响应后一次性写入
215 "called_at": {
216     "type": "TEXT",
217     "format": "ISO8601 UTC",
218     "definition": "API 调用发起时间戳，在内存中记录后收到响应时写入 DB，不提前写入避
免干扰响应时间计算"
219 },
220
221 "response_at": {
222     "type": "TEXT",
223     "format": "ISO8601 UTC",
224     "definition": "收到 LLM 响应的时间戳，与 called_at 一同在收到响应后写入"
225 },
226
227 "response_time_ms": {
228     "type": "INTEGER",
229     "format": "毫秒整数",
230     "definition": "LLM 纯响应时间（毫秒）= (response_at - called_at) × 1000，只
反映 LLM 的实际响应速度，排除 DB 写入、JSON 解析、Schema 校验等操作的耗时"
231 },
232
233 // — API 调用信息

```

```
234 "model_name": {
235     "type": "TEXT",
236     "constraint": "NOT NULL",
237     "allowed_values": ["gpt-4o", "claude-opus-4-6"],

```

```

238     "definition": "本次 API 调用实际使用的模型名称，与
extraction_runs.model_name 一致。v0.2 同一 run 内可能使用不同模型（如 repair 调用用
不同模型），预留此字段"
239 },
240
241     "http_status_code": {
242         "type": "INTEGER",
243         "allowed_values": [200, 400, 429, 500],
244         "definition": "API 调用返回的 HTTP 状态码。200 = 成功；429 = 超出速率限制；
500 = 服务器错误。与 step_status 配合使用：api_error 时记录具体状态码"
245     },
246
247     // — 步骤状态

```

```

248     // 写入顺序:
249     //   insert_step() → step_status = "processing" (占位)
250     //   update_step() → 解析完成后补充 parsed_output、llm_reasoning
251     //   update_step() → 校验完成后写入最终 step_status 和 error_detail
252     "step_status": {
253         "type": "TEXT",
254         "constraint": "NOT NULL",
255         "default": "processing",
256         "allowed_values": {
257             "processing": "初始占位，insert_step() 时写入",
258             "success": "JSON 解析通过 + Schema 校验通过",
259             "parse_error": "LLM 输出非合法 JSON，raw_response 已保存可供复盘",
260             "schema_error": "JSON 合法但字段不符合规范，如 enum 值超出允许范围",
261             "api_error": "API 返回非 200 状态码",
262             "timeout": "API 调用超时"
263         }
264     },
265
266     "error_detail": {
267         "type": "TEXT",
268         "definition": "step 级别的错误详情，比 run_status 粒度更细",
269         "examples": {
270             "parse_error": "LLM output is not valid JSON",
271             "schema_error": "字段: records -> 0 -> entity_type, 原因: 'short
peptide' is not one of ['peptide', 'protein', 'derivative', 'chimera',
'polymer', 'mixture', 'unclear']",
272             "api_error": "HTTP 429: Rate limit exceeded",
273             "timeout": "Timeout after 30s, threshold: 30s"
274         }
275     },
276
277     // — 系统元数据

```

```
278     "created_at": {
279         "type": "TEXT",
280         "default": "datetime('now')",
281         "definition": "数据库记录创建时间, SQLite 自动写入"
282     }
283 }
284 }
```

3.2 TraceLogger类实现

TraceLogger 封装 Trace 的全部写入逻辑，Text Agent 通过调用其方法完成记录，不直接操作 Trace 数据库。

方法	职责	调用时机
__init__()	生成 run_id, INSERT extraction_runs, run_status = running	Text Agent 开始时
insert_step()	收到 LLM 响应后立即 INSERT trace_steps, step_status = processing	收到 LLM 响应后立即
update_step()	逐步补充 trace_steps 字段，只更新传入的字段	解析完成后 / 校验完成后
update_run()	逐步补充 extraction_runs 字段，只更新传入的字段	业务写入完成后
finalize()	汇总 token，写入最终状态，UPDATE extraction_runs	完整性检查完成后
get_run_id()	返回当前 run_id，业务写入时存入 paper_record	写入业务数据库时

3.3 关键设计决策

决策一：Trace 使用独立的 SQLite 文件（hap_trace.db）

业务数据和 Trace 数据的生命周期不同。业务数据是长期积累的科研产出，需要严格维护；Trace 数据是执行过程日志，未来可能定期清理或归档。分开存储后两者可以独立备份、独立迁移，互不影响。同时避免 Trace 的高频写入对业务数据库造成锁竞争。

决策二：写入模式为 INSERT 占位 + 多次 UPDATE

TraceLogger 初始化时仅在内存中生成 run_id，调用 `start()` 后才 INSERT extraction_runs；收到 LLM 响应后立即 INSERT trace_steps（step_status = processing），后续各阶段通过 UPDATE 逐步补充字段。

决策三：called_at 和 response_at 均在收到响应后写入

两个时间戳都在收到 LLM 响应后用 time.time() 计算，收到响应后一次性写入 trace_steps

决策四：token 汇总在 finalize() 中从 trace_steps 聚合计算

total_tokens 系列字段由 finalize() 在 trace_steps 上执行 SUM 聚合计算后写入，不依赖调用方传入，确保统计结果准确。

决策五：prompt_system 和 prompt_user 完整保存，不截断

两个 Prompt 字段完整存储在 trace_steps 中

4.两张表的关系

4.1 extraction_runs 与 trace_steps 的关系

Trace 数据库由两张表组成，分别记录不同粒度的信息：

	extraction_runs	trace_steps
记录粒度	一次抽取任务（一篇论文）	一次 LLM API 调用
记录数量	1 篇论文 = 1 条记录	v0.1: 1 篇论文 = 1 条记录 v0.2 repair/分块: 1 篇论文 = N 条记录
主要内容	整体结果、token 汇总、完整性检查状态	Prompt 内容、原始响应、解析结果、响应时间
写入时机	TraceLogger 初始化时 INSERT finalize() 时最终 UPDATE	收到 LLM 响应后立即 INSERT 解析/校验完成后逐步 UPDATE
关联方式	run_id（主键）	run_id（外键，关联 extraction_runs）

4.2 与业务数据库的关联

Trace 数据库通过 run_id 与业务数据库关联，实现端到端溯源：

Code block

```
1  -- 端到端溯源查询示例
2  -- 给定一条 record_id，找到对应的完整 Trace
3
4  -- 步骤一：从业务数据库找到 run_id
5  SELECT extraction_run_id
6  FROM paper_record           -- hap_v01.db
7  WHERE record_id = 'P000001-R01';
```

```

8
9  -- 步骤二: 用 run_id 查整体执行结果
10 SELECT run_id, model_name, run_status,
11         total_tokens, records_extracted,
12         integrity_check_status
13 FROM extraction_runs          -- hap_trace.db
14 WHERE run_id = 'run_20240315_143022_abc123';
15
16 -- 步骤三: 查该 run 的每次 API 调用详情
17 SELECT step_index, model_name,
18        input_tokens, output_tokens,
19        response_time_ms, step_status,
20        llm_reasoning
21 FROM trace_steps              -- hap_trace.db
22 WHERE run_id = 'run_20240315_143022_abc123';

```

5.数据库初始化

提供 init_trace_db.py 一键初始化 Trace 数据库，包含建表和建索引全部操作。与业务数据库的初始化脚本分开，两者独立执行。

Code block

```

1  # init_trace_db.py
2  import sqlite3
3  import os
4
5  TRACE_DB_PATH = os.getenv('TRACE_DB_PATH', 'data/hap_trace.db')
6
7  def init_trace_database():
8      os.makedirs(os.path.dirname(TRACE_DB_PATH), exist_ok=True)
9      conn = sqlite3.connect(TRACE_DB_PATH)
10     conn.execute('PRAGMA foreign_keys = ON;')
11     conn.execute('PRAGMA journal_mode = WAL;')
12     _create_tables(conn)
13     _create_indexes(conn)
14     conn.commit()
15     conn.close()
16     print(f'Trace database initialized at {TRACE_DB_PATH}')
17
18 if __name__ == '__main__':
19     init_trace_database()

```

